

08-gRPC-ML-Serving

1 gRPC for ML Serving

Document Version: 1.0
Generated: December 4, 2025
Standard: gRPC Protocol Buffers for ML
Status: Industry Standard (Google)

1.1 Overview

gRPC (Google Remote Procedure Call) is a high-performance RPC framework using Protocol Buffers for serialization. For ML serving, gRPC provides:

- **Streaming:** Bidirectional streaming for real-time predictions
- **Performance:** Binary serialization, HTTP/2 multiplexing
- **Type Safety:** Protocol Buffer contracts
- **Interoperability:** Language-agnostic services

1.1.1 Key Features

- **Protocol Buffers:** Efficient binary serialization
- **HTTP/2:** Multiplexed connections
- **Streaming:** Client/server/bidirectional streams
- **Load Balancing:** Built-in client-side balancing
- **Deadline Propagation:** Request timeouts
- **Cancellation:** Server-side cancellation support

1.1.2 Primary Use Cases

1. **Model Inference:** Low-latency predictions
 2. **Streaming Predictions:** Real-time data streams
 3. **Batch Predictions:** Efficient batching
 4. **Service Integration:** Microservice communication
-

1.2 Authoritative References

- **gRPC Official:** <https://grpc.io>
 - **Protocol Buffers:** <https://protobuf.dev>
 - **gRPC Documentation:** <https://grpc.io/docs>
 - **Best Practices:** <https://grpc.io/docs/guides/performance-best-practices>
-

1.3 Format Structure

1.3.1 Protocol Buffer Definition

Real example (predict.proto):

```
syntax = "proto3";

package inference.v1;

option go_package = "github.com/example/inference/v1;inferencev1";
option java_multiple_files = true;

// Request for single prediction
message PredictRequest {
    string model_name = 1;
```

```

string model_version = 2;

// Input tensors as flattened arrays
map<string, TensorData> inputs = 3;

// Optional metadata
map<string, string> parameters = 4;
}

// Tensor data representation
message TensorData {
    repeated float float_val = 1;
    repeated int32 int_val = 2;
    repeated int64 int64_val = 3;
    repeated string string_val = 4;

    // Shape information
    repeated int64 tensor_shape = 5;

    // Data type
    DataType dtype = 6;
}

// Response with predictions
message PredictResponse {
    string model_name = 1;
    string model_version = 2;

    // Output tensors
    map<string, TensorData> outputs = 3;

    // Timing info
    int64 model_inference_ms = 4;
}

// Batch prediction request
message BatchPredictRequest {
    string model_name = 1;
    repeated PredictRequest requests = 2;
}

// Batch prediction response
message BatchPredictResponse {
    string model_name = 1;
    repeated PredictResponse responses = 2;
}

// Health check
message HealthCheckRequest {
    string service = 1;
}

message HealthCheckResponse {
    enum ServingStatus {
        UNKNOWN = 0;
        SERVING = 1;
        NOT_SERVING = 2;
    }
    ServingStatus status = 1;
}

// Supported data types
enum DataType {
    DTYPE_INVALID = 0;
    DT_FLOAT = 1;
    DT_DOUBLE = 2;
    DT_INT32 = 3;
    DT_INT64 = 4;
    DT_STRING = 5;
}

// Service definition
service PredictionService {

```

```

// Unary RPC: single request/response
rpc Predict(PredictRequest) returns (PredictResponse) {}

// Batch prediction
rpc BatchPredict(BatchPredictRequest) returns (BatchPredictResponse) {}

// Server-side streaming
rpc StreamPredict(PredictRequest) returns (stream PredictResponse) {}

// Client-side streaming
rpc StreamPredictBatch(stream PredictRequest) returns (PredictResponse) {}

// Bidirectional streaming
rpc StreamPredictBidi(stream PredictRequest) returns (stream PredictResponse) {}

// Health check
rpc Check(HealthCheckRequest) returns (HealthCheckResponse) {}
}

```

1.3.2 Protobuf Binary Format (Pseudo-Structure)

```

Wire Format Example:
Message: PredictRequest {
  model_name = "bert-classifier",
  model_version = "v1",
  inputs = { "input_ids": TensorData(...) }
}

Binary representation:
[0x0a 0x10] "bert-classifier"    // Field 1 (model_name) - length-delimited
[0x12 0x02] "v1"                // Field 2 (model_version) - length-delimited
[0x1a ...] <inputs data>        // Field 3 (inputs) - map

Tag encoding:
field_number << 3 | wire_type
Model_name (field 1): wire_type=2 (length-delimited)
Tag = 1 << 3 | 2 = 10 (0x0a)

```

1.4 Procedural Use-Cases

1.4.1 Use-Case 1: Unary Prediction Service

C++ Server Implementation:

```

#include <grpc++/grpc++.h>
#include "predict.grpc.pb.h"
#include <iostream>

class PredictionServiceImpl final :
  public inference::v1::PredictionService::Service {
public:
  grpc::Status Predict(
    grpc::ServerContext* context,
    const inference::v1::PredictRequest* request,
    inference::v1::PredictResponse* response) override {

    // Validate request
    if (request->model_name().empty()) {
      return grpc::Status(
        grpc::StatusCode::INVALID_ARGUMENT,
        "model_name is required"
      );
    }
  }
}

```

```

// Load model (cached)
auto model = LoadModel(request->model_name());
if (!model) {
    return grpc::Status(
        grpc::StatusCode::NOT_FOUND,
        "Model not found"
    );
}

try {
    // Extract inputs
    std::map<std::string, std::vector<float>> inputs;
    for (const auto& [key, tensor] : request->inputs()) {
        std::vector<float> values(
            tensor.float_val().begin(),
            tensor.float_val().end()
        );
        inputs[key] = values;
    }

    // Run inference
    auto start = std::chrono::high_resolution_clock::now();
    auto predictions = model->Predict(inputs);
    auto end = std::chrono::high_resolution_clock::now();

    // Populate response
    response->set_model_name(request->model_name());
    response->set_model_version(request->model_version());

    for (const auto& [key, values] : predictions) {
        auto& output_tensor = (*response->mutable_outputs())[key];
        for (float v : values) {
            output_tensor.add_float_val(v);
        }
    }

    // Set timing
    auto duration = std::chrono::duration_cast<
        std::chrono::milliseconds>(end - start);
    response->set_model_inference_ms(duration.count());

    return grpc::Status::OK;

} catch (const std::exception& e) {
    return grpc::Status(
        grpc::StatusCode::INTERNAL,
        std::string("Prediction failed: ") + e.what()
    );
}

}

grpc::Status Check(
    grpc::ServerContext* context,
    const inference::v1::HealthCheckRequest* request,
    inference::v1::HealthCheckResponse* response) override {

    response->set_status(
        inference::v1::HealthCheckResponse::SERVING
    );
    return grpc::Status::OK;
}

private:
    std::map<std::string, std::shared_ptr<Model>> model_cache_;

    std::shared_ptr<Model> LoadModel(const std::string& name) {
        if (model_cache_.find(name) == model_cache_.end()) {
            try {
                model_cache_[name] = std::make_shared<Model>(
                    "/models/" + name
                );
            } catch (...) {
                return nullptr;
            }
        }
    }

```

```

        }
    }
    return model_cache_[name];
}
};

int main() {
    PredictionServiceImpl service;

    grpc::ServerBuilder builder;
    builder.AddListeningPort(
        "0.0.0.0:50051",
        grpc::InsecureServerCredentials()
    );
    builder.RegisterService(&service);

    std::unique_ptr<grpc::Server> server(builder.BuildAndStart());
    std::cout << "Server listening on 0.0.0.0:50051" << std::endl;

    server->Wait();
    return 0;
}

```

1.4.2 Use-Case 2: Python Client

```

import grpc
from predict_pb2 import PredictRequest, TensorData
from predict_pb2_grpc import PredictionServiceStub

class MLServingClient:
    def __init__(self, host: str = "localhost", port: int = 50051):
        self.channel = grpc.aio.secure_channel(
            f"{host}:{port}",
            grpc.ssl_channel_credentials()
        )
        self.stub = PredictionServiceStub(self.channel)

    async def predict(
        self,
        model_name: str,
        inputs: dict,
        timeout: int = 10
    ) -> dict:
        """Send prediction request to server"""

        # Build request
        request = PredictRequest()
        request.model_name = model_name
        request.model_version = "v1"

        # Convert inputs to proto tensors
        for key, values in inputs.items():
            tensor = TensorData()
            tensor.float_val.extend(values)
            tensor.tensor_shape.extend([len(values)])
            request.inputs[key].CopyFrom(tensor)

        # Make RPC call
        try:
            response = await self.stub.Predict(
                request,
                timeout=timeout
            )

            # Parse outputs
            outputs = {}
            for key, tensor in response.outputs.items():
                outputs[key] = list(tensor.float_val)

            return {
                "model": response.model_name,
                "outputs": outputs,
                "latency_ms": response.model_inference_ms
            }

        except grpc.RpcError as e:
            print(f"RPC failed: {e.code()}, {e.details()}")
            raise

# Usage
import asyncio

async def main():
    client = MLServingClient()

    results = await client.predict(
        "bert-classifier",
        {"input_ids": [101, 2054, 2003, 2009, 102]}
    )

    print(f"Prediction: {results['outputs']}")
    print(f"Latency: {results['latency_ms']}ms")

asyncio.run(main())

```

1.4.3 Use-Case 3: Streaming Predictions

Server-Side Streaming (C++):

```
grpc::Status StreamPredict(
    grpc::ServerContext* context,
    const inference::v1::PredictRequest* request,
    grpc::ServerWriter<inference::v1::PredictResponse>* writer) override {

    auto model = LoadModel(request->model_name());
    if (!model) {
        return grpc::Status::NOT_FOUND;
    }

    // Stream multiple predictions
    for (int i = 0; i < 100; ++i) {
        // Simulate streaming data
        auto predictions = model->Predict(GetNextBatch());

        inference::v1::PredictResponse response;
        response.set_model_name(request->model_name());
        // ... populate response

        if (!writer->Write(response)) {
            return grpc::Status::CANCELLED;
        }
    }

    return grpc::Status::OK;
}
```

Client-Side Streaming (Python):

```
async def batch_predict_streaming(
    self,
    model_name: str,
    data_stream: AsyncIterator[dict]
):
    """Send multiple requests in stream"""

    async def request_generator():
        async for data in data_stream:
            request = PredictRequest()
            request.model_name = model_name
            # ... populate inputs
            yield request

    # Call bidirectional streaming
    responses = await self.stub.StreamPredictBidi(
        request_generator()
    )

    async for response in responses:
        yield {
            "model": response.model_name,
            "outputs": dict(response.outputs)
        }
```

1.5 Examples

1.5.1 Example 1: Generated Python Code

From protobuf:

```
python -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=. predict.proto
```

Generated files: - `predict_pb2.py` - Message classes - `predict_pb2_grpc.py` - Service stubs

1.5.2 Example 2: Proto File with Comments

```
syntax = "proto3";

package inference;

// Inference request with multiple input formats
message InferenceRequest {
    // Model identifier (required)
    // Format: "model-name:version" or just "model-name"
    string model_id = 1;

    // Input tensor data
    // Keys map to model input names (e.g., "input_ids", "attention_mask")
    map<string, Tensor> inputs = 2;

    // Request parameters (optional)
    // Supported parameters:
    // - "batch_size": override batching
    // - "timeout_ms": request timeout in milliseconds
    map<string, string> parameters = 3;
}

message Tensor {
    // Tensor shape (e.g., [32, 768] for batch of 32 embeddings)
    repeated int64 shape = 1;

    // Flattened tensor data
    oneof data {
        FloatData float_data = 2;
        IntData int_data = 3;
    }
}

message FloatData {
    repeated float values = 1;
}

message IntData {
    repeated int32 values = 1;
}

message InferenceResponse {
    // Output tensors
    map<string, Tensor> outputs = 1;

    // Metadata
    int64 latency_ms = 2;
    string version = 3;
}

service Inference {
    // Single prediction
    rpc Predict(InferenceRequest) returns (InferenceResponse);
}
```

1.5.3 Example 3: Docker Deployment

Dockerfile:


```
FROM ubuntu:20.04

RUN apt-get update && apt-get install -y \
    build-essential \
    cmake \
    protobuf-compiler \
    libgrpc++-dev

WORKDIR /app

# Copy proto files
COPY *.proto ./

# Generate gRPC code
RUN protoc --cpp_out=. --grpc_out=. *.proto

# Copy source and build
COPY . .
RUN g++ -std=c++17 -o server server.cpp \
    `pkg-config --cflags --libs grpc++` \
    predict.pb.cc predict.grpc.pb.cc

EXPOSE 50051

CMD ["/server"]
```

1.6 Performance Characteristics

Aspect	Value
Serialization Overhead	< 5% (vs JSON)
Message Size	60-70% smaller than JSON
Throughput	10K-100K req/sec
Latency (p99)	< 10ms (local)
Connection Reuse	Multiplexed over HTTP/2

1.7 Tools & Ecosystem

Tool	Description	URL
gRPC	Framework	https://grpc.io
grpcui	Web interface	https://github.com/fullstorydev/grpcui
grpcurl	CLI tool	https://github.com/fullstorydev/grpcurl
buf	Proto management	https://buf.build
Envoy	Proxy	https://www.envoyproxy.io

Navigation: [Back to Index](#) | [Previous: Hugging Face Hub](#) | [Next: KServe](#)